PRACTICAL – 13

Program 1: Generic Containers Using Box and Pair

Aim:

To implement generic classes Box<T> and Pair<A,B> in Java to store different types of values and demonstrate type safety. Also, create a generic method to return the first element of an array.

Code:

```
class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}

class Pair<A, B> {
    private A first;
    private B second;
```

```
    Pair(A first, B second) {
        this.first = first;
        this.second = second;
    }

    public void display() {
        System.out.println("First: " + first);
        System.out.println("Second: " + second);
    }
}

public class GenericDemo {

    public static <T> T getFirst(T[] array) {
        return array[0];
    }

    public static void main(String[] args) {

        Box<Integer> intBox = new Box<>();
        intBox.set(100);

        Box<String> strBox = new Box<>();
        strBox.set("Java Programming");

        System.out.println("Integer Box: " + intBox.get());
        System.out.println("String Box: " + strBox.get());
```

```
        Pair<String, Integer> pair =
                new Pair<>("Tirth", 20);

        pair.display();

        String[] names = {"Meet", "Tirth", "Priya"};

        System.out.println("First Element: "
                + getFirst(names));
    }
}
```

Output:

Integer Box: 100

String Box: Java Programming

First: Tirth

Second: 20

First Element: Meet

Program 2: Bounded Statistics Using Generics

Aim:

To implement a bounded generic method using <N extends Number> to calculate the average of Integer, Double and Long arrays.

Code:

```
public class BoundedStats {
```

```
    public static <N extends Number>
    double average(N[] values) {

        double sum = 0;

        for (N value : values) {
            sum += value.doubleValue();
        }

        return sum / values.length;
    }

    public static void main(String[] args) {

        Integer[] intValues = {10, 20, 30};

        Double[] doubleValues = {10.5, 20.5, 30.5};

        Long[] longValues = {100L, 200L, 300L};

        System.out.println("Integer Average: "
                + average(intValues));

        System.out.println("Double Average: "
                + average(doubleValues));

        System.out.println("Long Average: "
                + average(longValues));
```

```
    }
}
```

Output:

Integer Average: 20.0

Double Average: 20.5

Long Average: 200.0

Program 3: Generic Stack Using Array

Aim:

To implement a generic stack using an array in Java that supports push, pop and display operations for different data types and handles the empty stack condition.

Code:

```
class GenericStack<T> {

    private Object[] stack;
    private int top;

    GenericStack(int size) {
        stack = new Object[size];
        top = -1;
    }

    public void push(T value) {

        if (top == stack.length - 1) {
```

```
            System.out.println("Stack is Full");
            return;
        }

        stack[++top] = value;
    }

    @SuppressWarnings("unchecked")
    public T pop() {

        if (top == -1) {
            System.out.println("Stack is Empty");
            return null;
        }

        return (T) stack[top--];
    }

    public void display() {

        if (top == -1) {
            System.out.println("Stack is Empty");
            return;
        }

        System.out.println("Stack Elements:");

        for (int i = top; i >= 0; i--) {
```

```
            System.out.println(stack[i]);
        }
    }
}

public class GenericStackDemo {

    public static void main(String[] args) {

        GenericStack<Integer> intStack =
                new GenericStack<>(5);

        intStack.push(10);
        intStack.push(20);
        intStack.push(30);

        System.out.println("Integer Stack:");
        intStack.display();

        System.out.println("Popped: " + intStack.pop());

        GenericStack<String> strStack =
                new GenericStack<>(5);

        strStack.push("Java");
        strStack.push("Python");
        strStack.push("C++");
        System.out.println("String Stack:");
```

```
        strStack.display();

        System.out.println("Popped: " + strStack.pop());

        System.out.println("Empty Stack Test:");

        GenericStack<Integer> emptyStack =
                new GenericStack<>(3);

        emptyStack.pop();
    }
}
```

Output:

```
Integer Stack:
Stack Elements:
30
20
10
Popped: 30
String Stack:
Stack Elements:
C++
Python
Java
Popped: C++
Empty Stack Test:
Stack is Empty
```